

Exercise 01 Review and Preview

Chapter 1 of Beginning Software Engineering--Software Engineering from 20000 Feet

Software engineering is also a kind of engineering. We need to accomplish certain tasks. The following sections describe the steps to keep a software engineering project on track.

Requirement Gathering

- One of the first steps in a software project is figuring out the requirements.
- Find out what the customers want and what the customers need.
- Turn the wants and needs into requirements documents.
- Perform based on its value
- The change of software engineering always happens since it doesn't exist in physical ways.

High-Level Design

- Decisions about what platform to use, what data design to use
- Include information about the project architecture at a relatively high level
- Cover every aspect of the requirements but not too more details

Low-Level Design

- Includes the information about **how** the piece of the project should work.
- Discover interactions between different pieces of the project, which require changes or not.

Development

- Refine the low-level designs until they know how to implement those designs in code.
- Test it to find and remove as many bugs as they reasonably can.

Testing

- Perform different kinds of tests to guarantee work property with the other parts of the system.
- First test their own code
- Then testers who didn't write it test it.
- Then integrated into the rest of the project and test the whole system

A Swarm of Bugs

- A but often creates a new bug and sometimes the bug fix is incorrect.
- Can never be certain that no bugs.
- The bugs will propagate from high-level steps to low-level steps.

Deployment

- Allow some extra time in the project schedule to handle the sorts of completely unexpected complications.

Maintenance

- The users will find bugs when using the software.
- Fixed bugs or add more applications

Wrap-Up

- Perform a post-mortem
- Evaluate the project and decide what went right and what went wrong

Everything All at Once

- Many of basic tasks overlap.
- Put people to work with as much of the project that is ready to roll at any given moment.
- Sometimes need flow backward.

Chapter 1 of Textbook

1.1 What is Software Engineering

- Purpose: develop software-based systems to achieve business goals
- Differences between software engineering and programming
Software focuses on **understanding** the interaction between system-to-be and its users and the environment and **design**. Programming focuses on codes and the implementation of design
- What is software engineering
Understand business problems, inventing solutions, evaluating alternatives and making design tradeoffs and choices.
- Each part of software may be easy but the problem may be difficult
- Software engineering relies on our ability to think about space and time, processes and interactions between processes and structures.

1.1.1 Why Software Engineering Is Difficult

- Not only need knowledge about **software domain** but also **problem domain**
- Software is a **formal domain** while the real world is **informal**.
- Modeling the physical world and finding good abstractions, which is just **approximations**.
- Product line and development standardization are largely missing.

1.1.2 Book Organization

- Object-Oriented Software Engineering(OOSE)
- Techniques for problem understanding and specification
- Metrics for measuring the software process and product quality
- Advanced tools for software design
- Structured data representation using XML.
- Building blocks for complex software
- Service-oriented architectures and Web services.

1.2 Software Engineering Lifecycle

The common software development phases are as follows:

- Requirements Specification
Understanding the usage scenarios and deriving the **static** domain model
- Design
Assigning responsibilities to objects and specifying detailed **dynamic** of interactions under different usage scenarios.
- Implementation
Encoding the design in a programming language
- Testing
Testing individual components(unit testing) and the entire system(integrating testing)
- Operation and maintenance
Running the system and fixing bugs and adding new features.

The activities proceed in a sequential manner, called **waterfall process**.

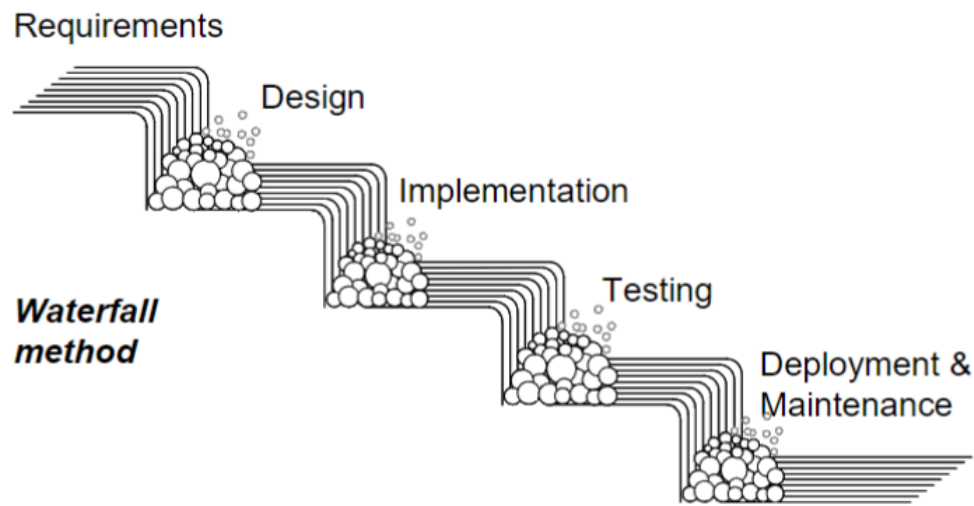


Figure 1-6: Waterfall process for software development.

However, there exists differences for software development

- Software is **intangible** and hard to visualize.
- Software is **complex** artifact.
- Software is **flexible** artifact.

The differences leads to **incremental and iterative** development methods:

- Break the big problem down into **smaller pieces(increments)** and **prioritize** them.
- In each iteration **progress** through the development in more depth
- Seek the customer **feedback** and change based on improved understanding.

The key is to progressively deepen the understanding or visualization of the target product by both advancing and retracting to earlier activities to rediscover its features.

The key tools to facilitate communication among develops:

- Modular design
break up the system in modules
- Symbol language
Like the symbols in mathematics to facilitate the readers' understanding of the material.
(Unified Modeling Language(UML))
- Project and product metrics
Tracking the work quality and progress towards completion
- Design heuristics
Also known as **patterns**. Create a design language for naming and describing the best practices.

Divide-and-Conquer approach is decomposing a problem into simpler ones, which is common in many fields. In software development it is embodied in **modularity**. In each module the source code can be written and maintained independently.

1.2.1 Symbol Language

What we use is the **short-term** memory, also called **working memory**, only temporarily memory for about 7 projects. So we organize information on higher levels of abstractions, which is called **chunking**.

The primary symbol language of this book is UML. It is used when documenting the completed design rather than for initial sketches.

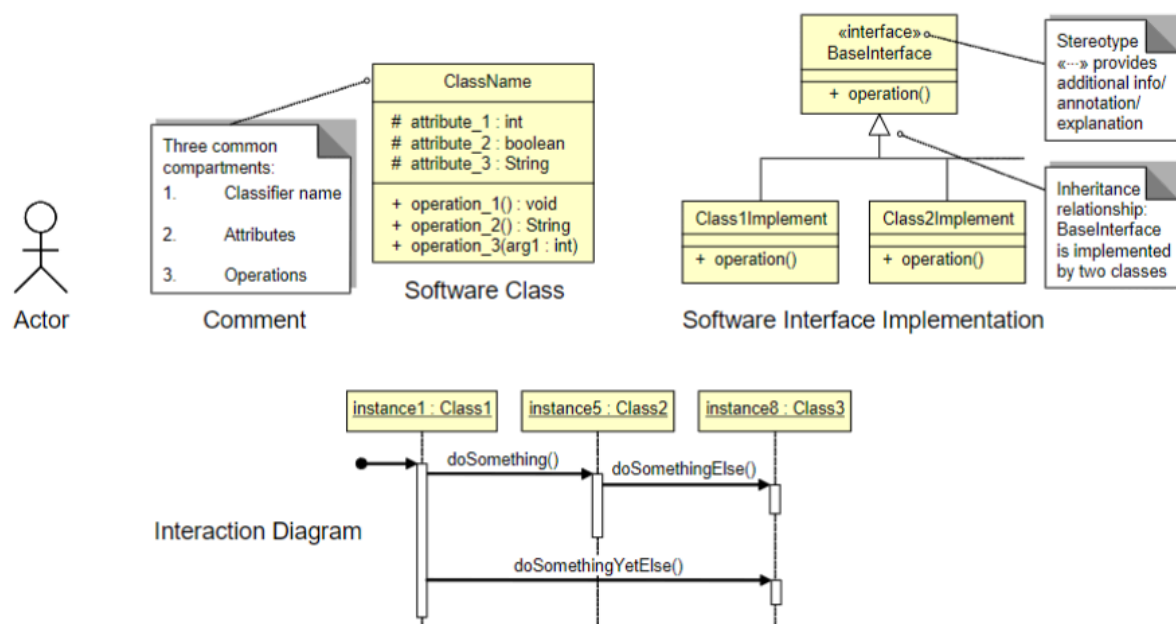


Figure 1-7: Example UML symbols for software concepts.

1.2.2 Requirement Analysis and System Specification

- Start at the **customer statement of work(customer statement of requirement)** or the **vision statement**
- First step is **requirements analysis** or **systems analysis**.

Understand the problem and delimit its scope, produce the system specification.

A popular technique for requirement analysis is **use case modeling**. The use case specify **what information must pass the boundary of the system** of a dialog. Each use-case name must include a **verb capturing the goal achievement**.

1.2.3 Object-Oriented Analysis and the Domain Model

Use cases consider the system as a **black box**. The next step is to **model the inside of the system** by **building the domain model** to show what the black box encloses.

Use cases elaborate the system's **behavioral characteristics** while the domain model details the system's **structural characteristics**.

We cannot delimit domain analysis to what the black box will envelop and we need to consider entities that **are both external and internal to the software-to-be**.

1.2.4 Object-Oriented Design

In software development, the key activity in the design phase is **assigning responsibilities** to software objects.

Design is the creative process of searching how to implement all of the customer's requirements.

A key problem of design is that we cannot know for sure if a design will work unless we **implement it and try it**.

The issues of software design include:

- Design quality evaluation
- Design for change
- Design for reuse

1.2.5 Project Effort Estimation and Project Quality Measurement

Incremental and iterative methods contribute a lot to project effort estimation and project quality measurement.

The **contiguous participation in all important decisions of customer** also affects the project effort estimation and project quality measurement.

Essence of Software Engineering: The SEMAT Kernel

- SEMAT founded in 2009 to solve immature SE practices; core goals: build a universal SE kernel and solid theoretical foundation.
- SEMAT Kernel: a universal, actionable thinking framework for SE, the common ground for method discussion and practice-independent assessment.
- Three core features: actionable (7 alphas with state checklists, non-linear collaborative network); extensible (modular practice separation, adaptable to diverse projects); practical (alpha cards for professionals' daily use).
- Core applications: 3-step iteration planning; scalable to all team sizes/SE lifecycle; balances fine-grained task tracking and holistic project assessment.

- Real-world use: adopted by enterprises like MunichRe, Fujitsu, KPN; taught in SE courses at KTH (Sweden).
- Method-agnostic: complements Agile/Waterfall/Scrum etc., aligns with Agile Manifesto; enables incremental legacy method migration and fixes project imbalances.
- Benefits: Teams – assess project health, optimize work; Industry – boost interoperability, transform method sharing; Academia – core course framework, research reference model.
- Vision: Redefine SE with solid theory; evolve with tech/requirements; serve as a unifying global SE framework, support legacy software maintenance.

Experience with SE

From my reading, I've learned that SE is far more than just programming—it is a systematic engineering discipline covering the entire software lifecycle, including requirements gathering, high/low-level design, development, testing, deployment, and maintenance. I recognize SE's unique challenges: software's intangibility, complexity, and flexibility make the linear waterfall model less practical, while incremental/iterative approaches are more adaptive to real-world needs. I also understand that SE practices are being standardized and refined to address immature practices, and that SE methods are complementary rather than mutually exclusive, with a focus on problem understanding, design tradeoffs, and holistic project assessment.

Expectation for the Course

I hope the course will:

- Deepen my understanding of core SE methodologies (e.g., use case modeling, UML, modular design) and how to apply them in different lifecycle phases;
- Provide practical insights into using frameworks like the SEMAT Kernel to assess project health, plan iterations, and balance fine-grained task tracking with holistic project management;
- Explore real-world case studies to master skills like requirement analysis, design decision-making, and bug/change management;
- Help me grasp the application of object-oriented software engineering and learn to make rational tradeoffs in SE practice.

Overall, I aim to move beyond theoretical knowledge to build actionable skills for solving practical SE problems and effectively applying SE principles to diverse projects.